



Lab Title: Sensitive Data Exposure

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 08/02/2026

Executive Summary

In this lab, I explored the issue of sensitive data exposure in a web application. I created a simple login system where usernames and passwords were sent over HTTP and stored in plain text. This made the data easy to see and steal during transmission. I was able to observe how login details could be exposed when no encryption was used. After exploiting the weakness, I secured the application by enabling HTTPS and hashing the passwords before storage. This lab helped me understand how poor data protection can expose users' information and how proper security measures can protect sensitive data.

Lab Objective

The objective of this lab was to help me understand how sensitive data can be exposed due to weak security practices. I aimed to learn how attackers can intercept unencrypted data and view sensitive information such as passwords. I also focused on learning how to protect this data by using secure communication (HTTPS) and password hashing. By completing this lab, I gained practical knowledge on how to safely handle and store sensitive user information.

Lab Environment

I performed this lab on my local machine using a PHP-based web application hosted on localhost. The environment included an Apache web server and basic HTML and PHP files such as login.html and process_login.php. I used this setup to simulate both insecure and secure login systems. This controlled environment allowed me to test the vulnerability, exploit it, and apply security fixes safely without affecting a real production system.

Task 1: Exploiting Sensitive Data Exposure

Step 1: Setting up the Vulnerable PHP Application

- Created a new folder ([Lab6_SensitiveDataExposure](#))

Session Actions Edit View Help

localhost/lab6_SecurityMisconfiguration

(raph@kali)-[/opt/lampp/htdocs]

```
$ mkdir lab6_SensitiveDataExposure
mkdir: cannot create directory 'lab6_SensitiveDataExposure': Permission denied
```

(raph@kali)-[/opt/lampp/htdocs]

```
$ sudo mkdir lab6_SensitiveDataExposure
[sudo] password for raph:
```

(raph@kali)-[/opt/lampp/htdocs]

```
$ ls
applications.html      Lab4_CommandInjection
bitnami.css            lab6_SensitiveDataExposure
dashboard              lab8_BrokenAccessControl
favicon.ico            lab9_SecurityMisconfiguration
img                    test.php
index.php              vulnerable_app
lab10_xss              webalizer
lab17_Insecure_Deserialization
Lab2_HTMLInjection    week2_php_lab
```

Apache/2.4.56 (Unix) OpenSSL/1.1.1t PHP/8.2.4 mod_perl/2.0.12 Perl/v5.34.1

(raph@kali)-[/opt/lampp/htdocs]

```
$
```

- Inside the folder, I created two files
 - login.html (for login form)
 - process_login.php (to handle login requests and expos sensitive data)

```
(raph@kali)-[/opt/lampp/htdocs/lab6_SensitiveDataExposure]
$ sudo nano login.html
(raph@kali)-[/opt/lampp/htdocs/lab6_SensitiveDataExposure]
$ sudo nano process_login.php
(raph@kali)-[/opt/lampp/htdocs/lab6_SensitiveDataExposure]
$ ls
login.html  process_login.php
(raph@kali)-[/opt/lampp/htdocs/lab6_SensitiveDataExposure]
$
```

Step 2: Implemented the Vulnerable Code

- login.html: This file will contain a simple login form where users can enter their username and password.

```
raph@kali: /opt/lampp/htdocs/lab6_SensitiveDataExposure
Session Actions Edit View Help
GNU nano 8.6 login.html *
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Sensitive Data Exposure Example</title>
</head>
<body>
  <h1>Login Form</h1>
  <form action="process_login.php" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
    <button type="submit">Login</button>
  </form>
</body>
</html>

localhost
Apache/2.4.56 (Unix) OpenSSL/1.1.1f PHP/8.2.4 mod_perl/2.0.12 Perl/v5.34.1
```

- process_login.php: This PHP file will simulate the process of logging in. it will take user credentials and store them in plain text.

```

raph@kali: /opt/lampp/htdocs/lab6_SensitiveDataExposure
Session Actions Edit View Help
GNU nano 8.6 process_login.php *
<?php
// Simulate a database with hardcoded username and password
$stored_username = "user";
$stored_password = "password123"; // Stored as plain text (vulnera

// Capture user input
$user_input_username = $_POST['username'];
$user_input_password = $_POST['password'];

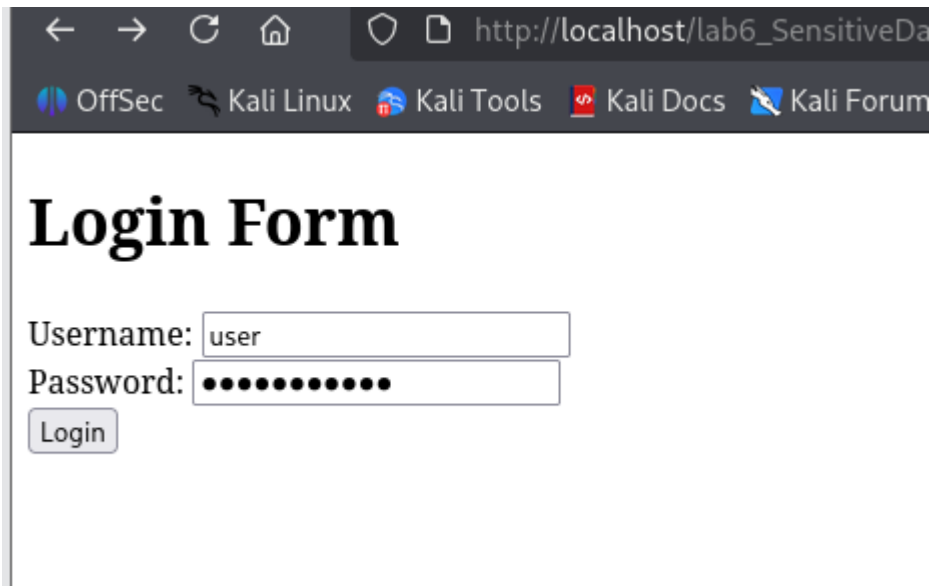
// Simulate a login check
if ($user_input_username == $stored_username && $user_input_passwo
    echo "<h2>Welcome, $user_input_username!</h2>";
    echo "<p>Your sensitive data is now exposed (in plaintext). Th
} else {
    echo "<h2>Invalid credentials. Try again.</h2>";
}
?>
Error 403

localhost
Apache/2.4.56 (Unix) OpenSSL/1.1.1t PHP/8.2.4 mod_perl/2.0.12 Perl/v5.34.1

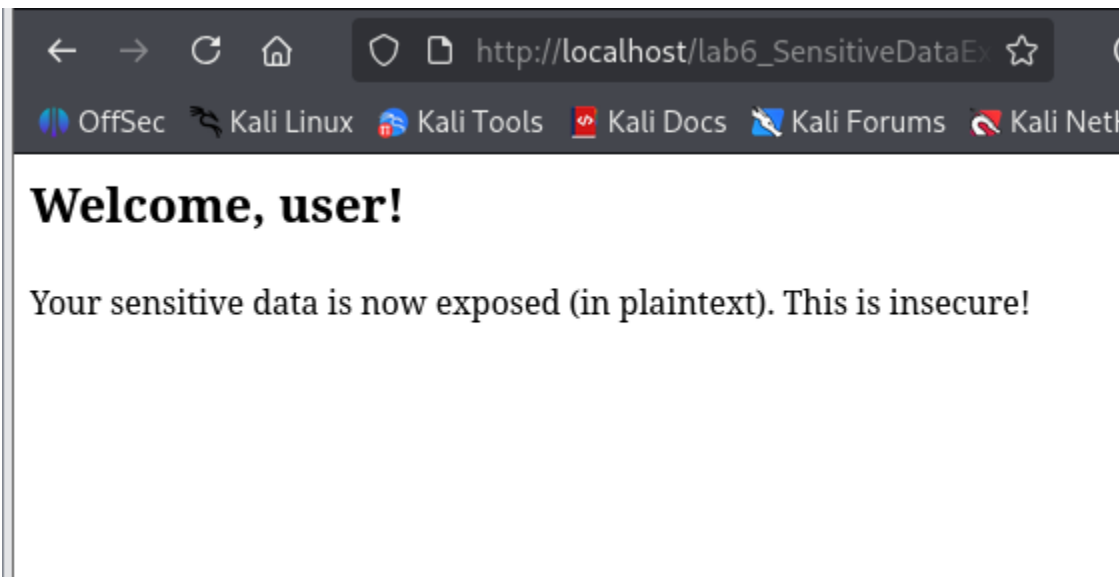
```

Step 3: Exploiting the Vulnerability

In this step, I opened the login.html file in a web browser and submitted the login form using the correct credentials (username: **user**, password: **password123**). After a successful login, the application displayed sensitive information in plain text, showing that the password was not protected.

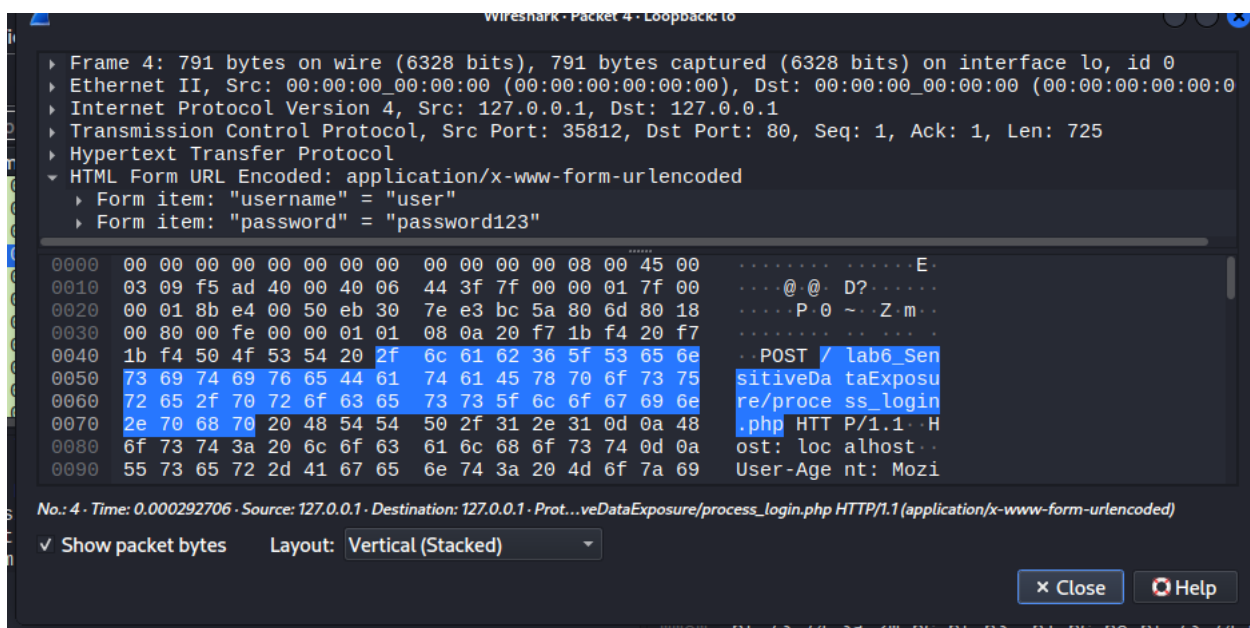
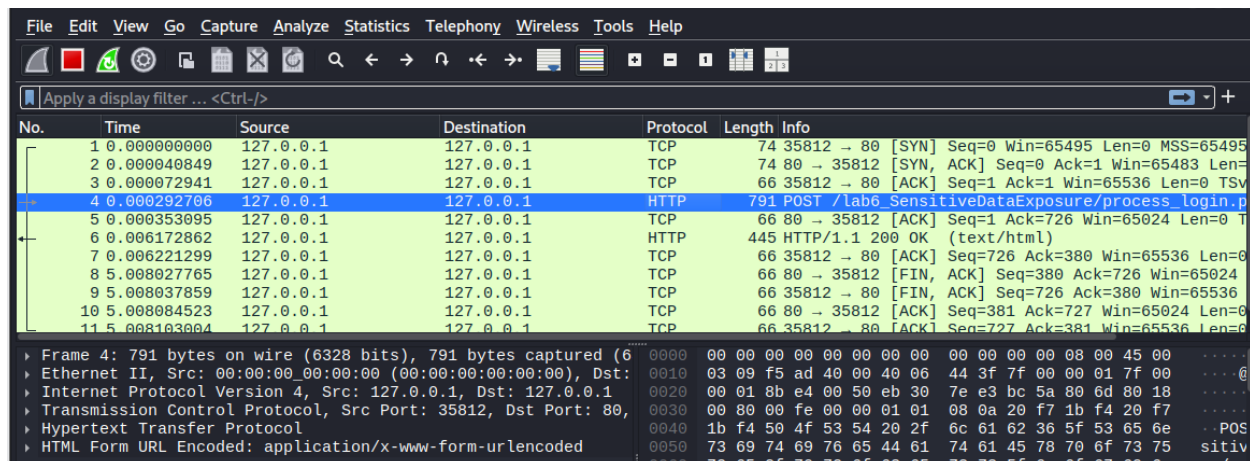


A screenshot of a web browser window. The address bar shows the URL `http://localhost/lab6_SensitiveDa`. The browser's bookmark bar contains links to OffSec, Kali Linux, Kali Tools, Kali Docs, and Kali Forum. The main content area displays a login form titled "Login Form". The form has two input fields: "Username:" with the text "user" and "Password:" with masked characters (dots). Below the password field is a "Login" button.



A screenshot of a web browser window after a successful login. The address bar shows the URL `http://localhost/lab6_SensitiveDataEx`. The browser's bookmark bar contains links to OffSec, Kali Linux, Kali Tools, Kali Docs, Kali Forums, and Kali NetH. The main content area displays a welcome message: "Welcome, user!" followed by the text "Your sensitive data is now exposed (in plaintext). This is insecure!"

I also tested how the data was transmitted over the network. Since the application was using HTTP instead of HTTPS, the login details were sent without encryption. By intercepting the traffic using tools like Wireshark, I was able to see the username and password in plain text. This confirmed that the application was vulnerable to sensitive data exposure.



Reflection Questions:

- What are the risks of transmitting sensitive data over HTTP?

When sensitive data is sent over HTTP, it is not encrypted. This means anyone monitoring the network can see information like usernames and passwords. It can lead to data theft, unauthorized access, and loss of user privacy.

- How could an attacker exploit the lack of encryption during the transmission of sensitive data?

An attacker can intercept the network traffic using tools such as Wireshark or Burp Suite. Since the data is sent in plain text, the attacker can easily read and steal login credentials and then use them to access the system without permission.

Task 2: Mitigate Sensitive Data Exposure

Step 1: Secure Password Storage Using Hashing

To secure password storage, I used password hashing instead of saving passwords in plain text. By hashing the passwords, even if someone gains access to the stored data, they cannot easily read or use the actual passwords.

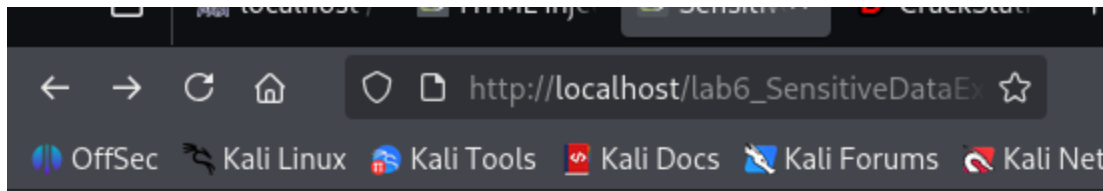
- I Modified the process_login.php file to hash the password using password_hash() and then I verified it using password_verify()

```
GNU nano 8.6 process_login.php
<?php
// Securely store the password using bcrypt
$stored_username = "user";
$stored_password_hash = '$2y$10$8N5f6eQmJ0nExI0ktHJ9euX0V/qwjo7ejj64ZPa1cxL2w5Xhbn3K'; // password123 hashed using bcrypt

// Capture user input
$user_input_username = $_POST['username'];
$user_input_password = $_POST['password'];

// Check if the user input matches the stored username and the hashed password
if ($user_input_username == $stored_username && password_verify($user_input_password, $stored_password_hash)) {
    echo "<h2>Welcome, $user_input_username!</h2>";
    echo "<p>Your sensitive data is now protected with hashing and secure transmission.</p>";
} else {
    echo "<h2>Invalid credentials. Try again.</h2>";
}
?>
```

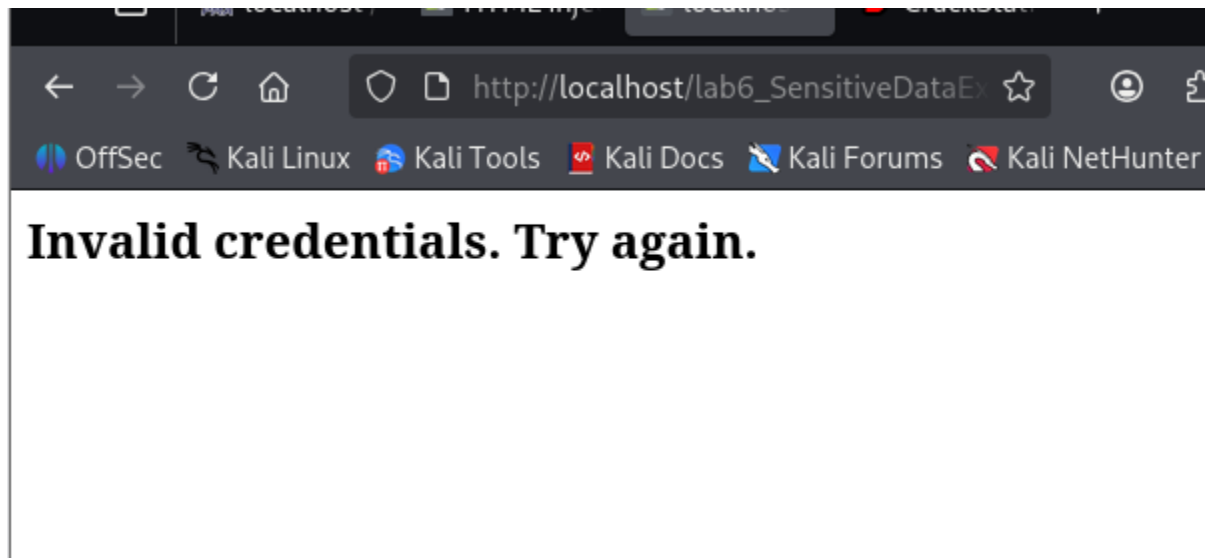
Step 2: Test the Mitigated Version



Login Form

Username:

Password:



Reflection Questions:

- How does using HTTPS protect the sensitive data transmitted between the client and server?

HTTPS encrypts the data sent between the client and the server. This makes the information unreadable to anyone who tries to intercept it, which helps prevent attackers from stealing sensitive data like usernames and passwords.

- **Why is it important to hash passwords before storing them?**

Hashing converts passwords into an unreadable format before storing them. This means that even if an attacker gains access to the stored data, they cannot easily recover the original passwords.

- **Can you think of any additional steps to improve the security of sensitive data in this application?**

Other security measures include using strong password policies, adding input validation, implementing account lockout after multiple failed login attempts, and keeping the server and software up to date.

Key Takeaway

Sensitive data must always be protected during transmission and storage. Using HTTPS encrypts data while it is being sent, and hashing passwords prevents attackers from reading them even if the system is compromised. Proper data protection greatly reduces the risk of sensitive information being exposed.

Conclusion

In conclusion, this lab helped me understand how sensitive data can be exposed when proper security measures are not in place. I learned that transmitting data over HTTP and storing passwords in plain text is very risky. By implementing HTTPS and password hashing, I was able to protect user information more effectively. This lab showed me the importance of securing sensitive data to prevent attacks and protect users' privacy.